Discover

Release decisions, ownership and evidence

Session 1 / lifecycle, evaluation and governance
16 slides / offline labs + release review

LLMOps controls behavior across releases

> Quality and safety need baselines, owners and release gates.

> Latency, failures and tokens need measured operating limits.

> Every change needs evidence, an approver and a rollback path.

Lifecycle / inventory before promotion

> Record model/provider ID, owner, purpose and approval state.

> Benchmark candidates on one frozen dataset and configuration.

> Check compatibility; retain a rollback bundle before retirement.

Version the release bundle and name its owners

Corpus / data owner

Source hashes + chunk/embedding version

Prompt / prompt owner

Template + parameters + approval record

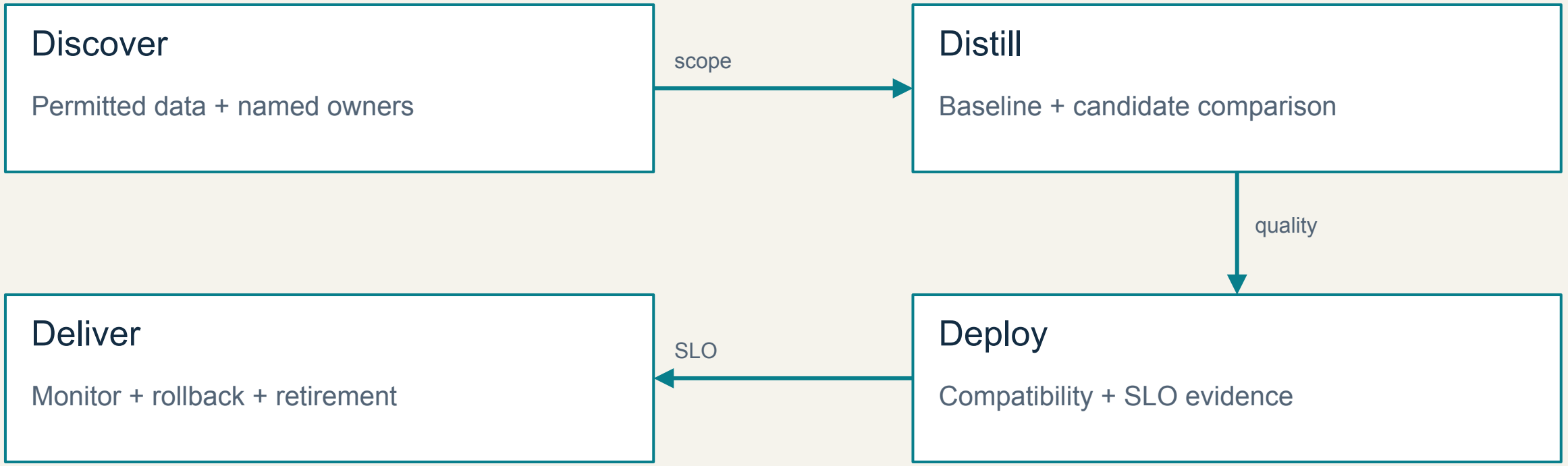
Evaluation / reviewer

Dataset + rubric + baseline + run ID

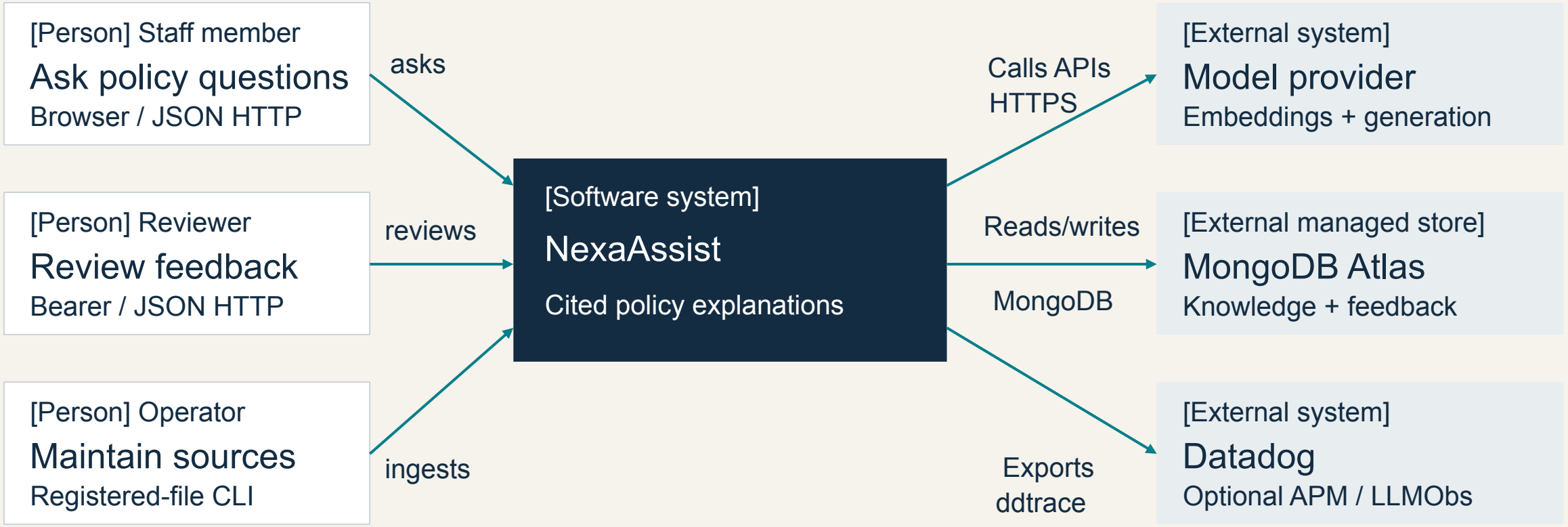
Runtime / service owner

Model/provider + configuration version

Four phases / explicit release decisions

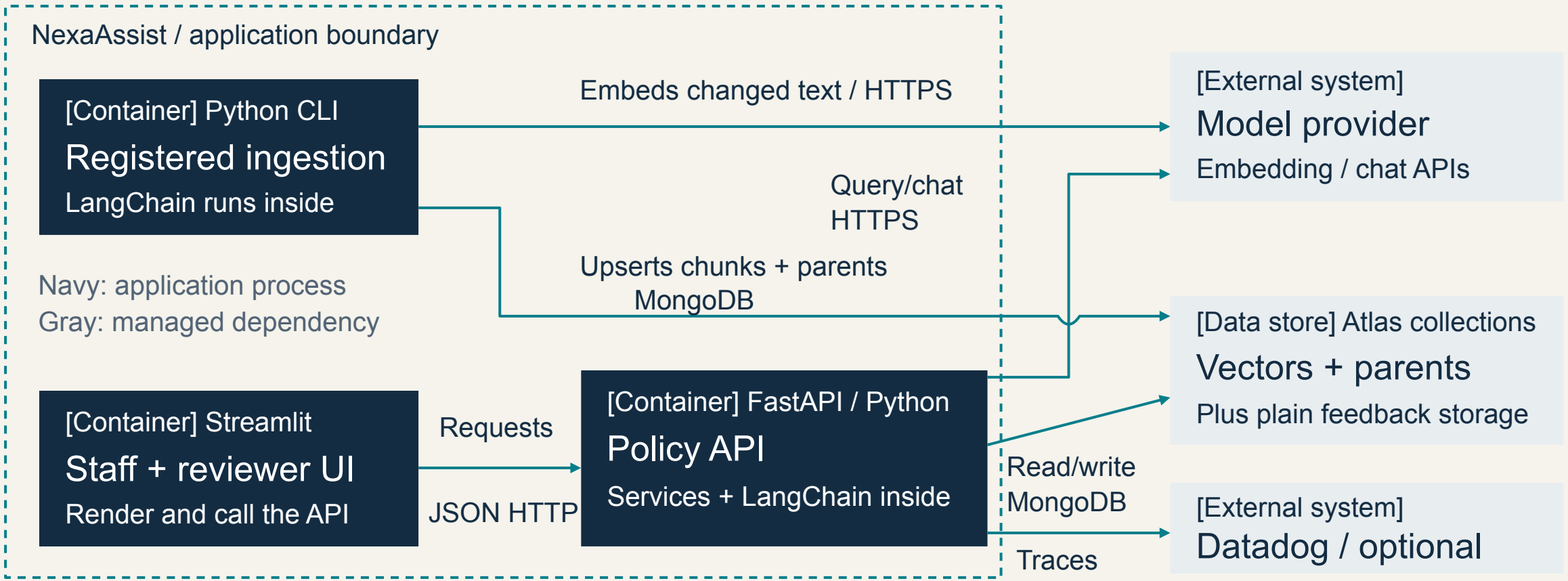


C4 / system context

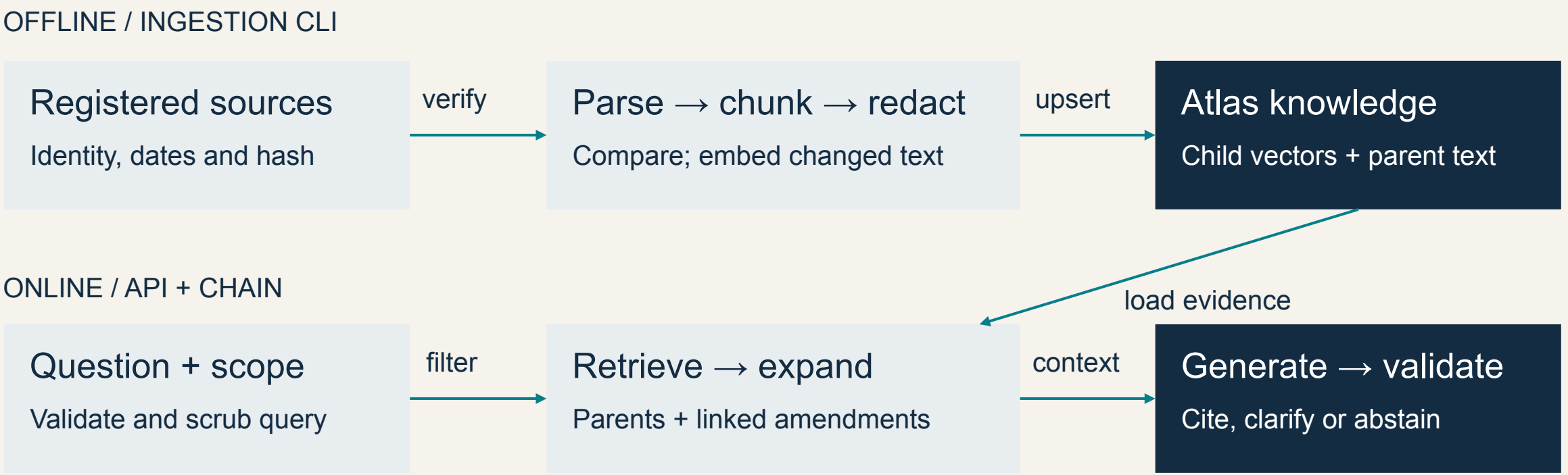


White: people · Navy: system in scope · Gray: external systems · Arrows name the initiating call.

C4 / containers and ownership

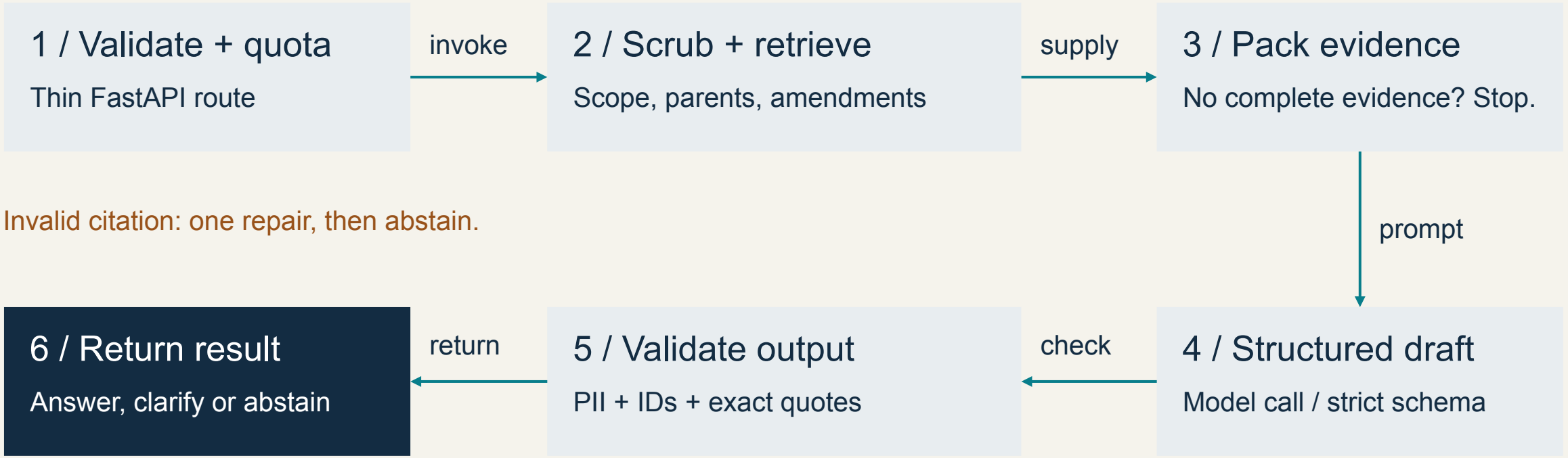


Two paths share one knowledge contract



Shared contract: source hash, policy date, embedding version and recoverable release identity.

One answer / request and failure flow



Invalid citation: one repair, then abstain.

Numbered boxes show the main path. Early abstention and the bounded repair are explained in notes.

Governance / three independent checks

Freshness: does indexed evidence match the approved source?

Permission: may this scope access and disclose the content?

Audit: can a reviewer recover versions and the decision?

EvalOps / freeze a trustworthy baseline

Dataset: input + expected_output + context + expected_status

Cases: normal, refusal, date boundary, adversarial

Compare: same dataset, rubric and release identities

Reference checks + a calibrated judge + human review.

Record disagreements; keep candidate tuning apart from the holdout.

Practical / validate gates, then decide

> Run: python -m training.session1.check_gates evaluation/gates.json

> In a temporary copy: set faithfulness_min=1.5; expect INVALID.

> Valid + status=placeholder means HOLD for production approval.

Code / reject changed source bytes

```
def _verify_source(path: Path, entry: dict[str, Any]) -> str:
    """Refuse a file whose bytes differ from the register's recorded sha256."""
    digest = hashlib.sha256(path.read_bytes()).hexdigest()
    if digest != entry["sha256"]:
        raise ValueError(f"{entry['doc_id']}: file sha256 does not match the document register")
    return str(path)
```

A source must match the register before parsing or embedding.

Practical / prove the source boundary

```
uv run --no-sync python -m training.showcase --session 1
```

Predict → run: 8 originals verify; a changed copy is refused.

Submit: document ID, hash error, owner and HOLD decision.

Practical / review a provider change

Complete inventory: prompt/model/config/data versions + owners.

Candidate lacks quality and compatibility evidence → HOLD.

Name approver, rollback bundle, trigger and retirement condition.

Session 1 / release-readiness exit gate

> Hand in: inventory, baseline cases and two failure transcripts.

> Defend: owner, evidence, HOLD or promote, rollback target.

> Pass the exercise; keep unmeasured production gates open.